Comparación de técnicas de recuperación de información en un sistema RAG educativo

Apéndices técnicos C, D y E

Andrés González Sánchez

2026-10-14

Tabla de contenidos

C. Configuración experimental 2
D. Resultados y material complementario 3
D.1. Contabilización de tiempos, tokens e intentos 4
E. Reproducción 4
E.1. Paquete de reproducción y requisitos 4
E.2. Reproducción mediante notebooks 5
E.3. Nueva inferencia: otra ejecución 6
E.4. Trazabilidad del experimento final 7
E.5. Corpus, figuras y render del apéndice 9

Este documento complementa la memoria de analista **Comparación de técnicas de recuperación de información en un sistema RAG educativo**. Reúne la configuración experimental (C), la localización de los resultados y registros operativos (D) y los procedimientos de reproducción (E). La numeración continúa los apéndices A y B incluidos en la memoria. Puede consultarse de forma independiente.

Las tablas de resultados y el inventario de preguntas se encuentran en la memoria de analista. Los antecedentes experimentales se conservan en un suplemento separado del paquete de trabajo y no se suman a la muestra final.

C. Configuración experimental

Tabla 1: *Configuraciones de retrieval comparadas*

Condición	Configuración principal	Profundidad y desempate
BM25	Tokenización `[a-z0-9]+`, $k_1 = 1,5$, $b = 0,75$	top-10; score descendente e identificador de chunk
Dense	`intfloat/e5-base-v2`, embeddings normalizados de 768 dimensiones	top-10; producto escalar exacto e identificador de chunk
RRF	Fusión de los top-10 de BM25 y dense, $k_0 = 60$	top-10; score RRF descendente e identificador de chunk
RRF + reranking	`cross-encoder/ms-marco-MiniLM-L6-v2` sobre los diez candidatos de RRF	conserva diez candidatos; score del cross-encoder e identificador de chunk

Para la recuperación densa se utilizó E5-base-v2, con los prefijos `query:` y `passage:`, longitud máxima de 512 tokens, lotes de 16 y búsqueda exhaustiva sobre NumPy. Para la reordenación se utilizó MiniLM-L6-v2, también con longitud máxima de 512 tokens y lotes de 16, en ejecución CUDA. Las revisiones exactas de los modelos se conservan en los metadatos de ejecución. La elección de modelos y parámetros no se hizo después de observar las métricas.

Tabla 2: *Contrato final de generación y evaluación*

Componente	Configuración final
Preguntas	26 preguntas base y 52 formulaciones: 17 preguntas base de respuesta, cinco de corrección y cuatro de abstención
Generadores	`gemma4:e4b`, `qwen3.5:4b`, `qwen3.5:9b`
Decodificación	Temperatura 0, semilla 20260902, 300 tokens de salida y contexto de 8192 tokens, según el adaptador
Prompt generativo	`rag_answer_prompt_v1`, cinco fragmentos y hasta tres intentos totales por celda
Evaluación válida	`ragas_rubric_v3.2`, instrumento ordinal propio, dos jueces y lotes de diez
Jueces	A: `gpt-6-sol`/`openai-codex`. B: `mimo-v2.6-pro`/`opencode-go`
Arbitraje y cierre	`gpt-6-astra`, jueces anonimizados y validación humana por criterio discrepante
Agregación	Media dentro de cada pregunta base y macro entre preguntas base, con soporte común exacto según contraste
Incertidumbre	Bootstrap de 5.000 réplicas, semilla 20260928 e IC puntuales
Fuentes de respuestas	Doce archivos y sus registros de fallos bajo `outputs/local_runs/generation_scaling_v3/`

Las configuraciones y revisiones exactas de recuperación están en `outputs/local_runs/explicitness_v3/configuration.json`. Las opciones de decodificación se conservan en el adaptador `scripts/run_rag_generation.py` y en el protocolo con sus enmiendas. No se dispone de una captura independiente de todas esas opciones en cada metadato histórico. La configuración del código no sustituye los metadatos ausentes de una ejecución.

D. Resultados y material complementario

La memoria emplea los resultados corregidos de recuperación en `outputs/local_runs/academic_corrections_20260930/retrieval/` y de comparación de generadores en `capacity/release_run1/`, dentro de esa misma raíz de correcciones. Los CSV distinguen el agregado de 26 preguntas base, la sensibilidad de veinte con evidencia de grado 2, las tareas de contenido y abstención y los soportes comunes.

La comparación final por recuperador y los registros operativos se encuentran en `outputs/local_runs/final_experiment_20260930/analysis/`:

- `per_retriever.csv`: medias y denominadores por criterio.

- `paired_contrasts.csv`: 72 contrastes entre recuperadores.
- `availability.csv` y `support_cells.csv`: disponibilidad y casos comparables.
- `operational_cells.csv`: 624 registros individuales, con las 604 respuestas aceptadas y los veinte fallos separados por estado.
- `operational.csv`: doce resúmenes por generador y recuperador.
- `operational_summary.csv`: 24 resúmenes desglosados por estado de aceptación o fallo.

Estos archivos se derivan sin modificar las fuentes congeladas. La comprobación de cifras se realiza contra artefactos externos al documento, no contra otra copia de sus tablas.

El suplemento `antecedentes_experimentales.qmd`, conservado en el paquete de trabajo, reúne resultados previos, instrumentos, auditorías y el benchmark controlado de recuperación. **Su muestra y sus escalas no se incorporan a los resultados finales.** Permite revisar el desarrollo del diseño sin mezclar evidencias de campañas distintas.

D.1. Contabilización de tiempos, tokens e intentos

Los campos de entrada y salida expresan tokens. Los tiempos y tokens registrados acumulan los intentos de cada unidad. No se añade otra vez el consumo del registro de intentos (*ledger*). Las respuestas aceptadas suman **614 intentos** y los veinte fallos, **sesenta**.

Las aceptadas conservan totales registrados, pero no un desglose numérico completo por intento que permita reconstruirlos independientemente. Los fallos sí conservan ese desglose. Solo se recuperaría un total ausente si todos los intentos declarados tuvieran datos numéricos. Esta regla no implica que se haya detectado una ausencia ni sustituido un total registrado.

Los tiempos corresponden a generación y acumulan los reintentos. No representan latencia controlada de extremo a extremo ni incluyen por sí solos el coste de recuperación o evaluación. Los tokens tampoco constituyen una estimación monetaria.

E. Reproducción

E.1. Paquete de reproducción y requisitos

La reproducción requiere el paquete de entrega completo, con los datos, el código y los resultados congelados. Los PDF están separados del material ejecutable. Disponer únicamente de los documentos no permite repetir los análisis.

```
v2_entrega_final/
├── documentos/       PDF de la entrega
├── reproduccion/     fuentes y paquete ejecutable
│   ├── notebooks/    recorrido de reproducción
│   ├── scripts/      implementación de los análisis
│   ├── data/         corpus, preguntas y evidencias
│   ├── config/       contratos y configuraciones
│   ├── outputs/      fuentes y resultados congelados
│   ├── environments/ entornos de inferencia
│   ├── docs/         protocolos y validadores
│   └── tests/        pruebas del pipeline
└── verification/     informes y cuadernos ejecutados
```

Las rutas de código y datos de este documento son relativas a **reproduccion/**. Esta carpeta contiene también las fuentes QMD, pyproject.toml y uv.lock. No se necesitan las carpetas originales v1 o v2.

El entorno general requiere Python 3.14 o superior y uv. Desde la raíz de la entrega se instala el entorno y se abre Jupyter Lab:

```
cd reproduccion
uv sync --locked --group dev
uv run --no-sync jupyter lab notebooks
```

En Jupyter debe seleccionarse el kernel del entorno del proyecto. Los comandos siguientes se ejecutan desde reproduccion/. La recuperación neuronal y la generación tienen entornos separados bajo environments/ y requisitos adicionales de modelos y servicios.

E.2. Reproducción mediante notebooks

Los cinco notebooks de reproduccion/notebooks/ constituyen la vía principal para reproducir los resultados. Presentan las fuentes, ejecutan los análisis mediante los scripts incluidos y muestran las tablas obtenidas. Se recomienda seguirlos en este orden:

1. **00. Entorno, corpus y dataset.** Comprueba los documentos de origen, la fragmentación y las unidades del experimento. Archivo: `00_entorno_corpus_y_dataset.ipynb`.
2. **01. Preguntas, evidencias y qrels.** Examina las dos formulaciones, las conductas esperadas y los grados y procedencia de la relevancia. Archivo: `01_preguntas_evidencias_y_qrels.ipynb`.
3. **02. Recuperación y explicitud.** Recalcula las métricas y el contraste explícita–implícita a partir de rankings y qrels congelados. Archivo: `02_recuperacion_y_explicitud.ipynb`.
4. **03. Respuestas, evaluación y decisiones.** Comprueba cobertura, fallos y procedencia de las puntuaciones. Permite leer una respuesta junto a los fragmentos recuperados. Archivo: `03_respuestas_evaluacion_y_decisiones.ipynb`.
5. **04. Comparaciones finales y verificación.** Ejecuta el reanálisis completo, compara los resultados con las referencias y muestra las tablas finales de calidad, disponibilidad y tiempos. Archivo: `04_comparaciones_finales_y_verificacion.ipynb`.

Cada cuaderno puede ejecutarse de arriba abajo con Restart Kernel and Run All, sin depender de las variables de otro notebook. Los cuadernos 00, 01 y 03 leen las fuentes y hacen comprobaciones en memoria. El 02 recalcula recuperación y explicitud. El **04 reproduce los 35 CSV** del análisis completo y exige que coincidan byte a byte con las referencias congeladas. Puede ejecutarse directamente para comprobar los resultados numéricos de la entrega.

Los cuadernos de análisis crean destinos nuevos con prefijo REPLAY_ bajo outputs/local_runs/final_experiment_20260930/. Repetir su ejecución no sobrescribe las fuentes, las decisiones ni los documentos. Los scripts mantienen una única implementación de las métricas y de los contrastes estadísticos.

La colección completa también puede ejecutarse desde consola:

```
uv run --no-sync python notebooks/verify_notebooks.py
```

El verificador utiliza un kernel nuevo para cada cuaderno, comprueba la ejecución de todas las celdas y guarda cinco copias ejecutadas y un informe en un destino REPLAY_ nuevo. Las copias de la comprobación de entrega y su informe están en verification/notebooks/ y verification/notebooks_validation.json, respecto a la raíz del repositorio. Las instrucciones de uso se encuentran en notebooks/README.md.

Como alternativa, el mismo reanálisis completo y la validación específica del apéndice pueden lanzarse sin abrir Jupyter:

```
uv run --no-sync python scripts/reproduce_offline.py \
  --run-id tribunal_01
uv run --no-sync python docs/validar_apendices.py
```

La ejecución por consola requiere un identificador nuevo en cada repetición. La validación específica comprueba recuentos, procedencia, integridad de las fuentes y correspondencia entre este apéndice y su PDF. La opción `--data-only` omite la comprobación documental y permite auditar los datos antes del render. Los validadores editoriales históricos de otras versiones no son el criterio de aceptación de esta entrega.

Este procedimiento reproduce el análisis de la evidencia incluida. No vuelve a ejecutar los modelos, no reconstruye juicios humanos ni constituye una nueva campaña. Las respuestas y las decisiones de referencia permanecen congeladas.

E.3. Nueva inferencia: otra ejecución

El adaptador de explicitud carga parejas existentes. No redacta preguntas ni adjudica su equivalencia. Una nueva recuperación neuronal se ejecuta por separado del reanálisis offline:

```
OUT=outputs/local_runs/NUEVO_IR_V3
test ! -e "$OUT" || exit 1
uv run --project environments/dense --no-sync python \
  scripts/explicitness_campaign_v3.py \
  --output "$OUT" --top-k 10
```

Una nueva condición de generación requiere rankings, corpus, golden set y resolución de premisas explícitos:

```
IR=outputs/local_runs/explicitness_v3
GOLD=data/golden
OUT=outputs/local_runs/NUEVA_GENERACION
test ! -e "$OUT" || exit 1
uv run --project environments/rag --no-sync python \
  scripts/run_rag_generation.py \
  --experiment-id gen_replay_v3_bm25_qwen35_4b \
  --retrieval-condition bm25 \
  --retrieval-run "$IR/runs/explicitness_v3_bm25.jsonl" \
  --chunks data/processed/chunks_v1.jsonl \
  --golden "$GOLD/variants/generation_golden_v3.jsonl" \
  --resolutions "$GOLD/answerability_resolutions_v1_22q.json" \
  --model qwen3.5:4b --top-k 5 \
  --prompt-version rag_answer_prompt_v1 \
  --output "$OUT/rag_bm25_top5_qwen35_4b.jsonl" \
  --metadata "$OUT/rag_bm25_top5_qwen35_4b_metadata.json"
```

Estas recetas requieren los modelos y servicios correspondientes y **no se ejecutan como parte de la validación documental**. La CLI opera por condición, no ofrece un runner factorial intercalado. Otra campaña debe registrar el orden real, la identidad de los pesos y el entorno. Temperatura cero y semilla fija no garantizan igualdad entre ejecuciones. Las decisiones humanas anteriores solo pueden reutilizarse si coinciden pregunta, respuesta, referencia y evidencia completas.

E.4. Trazabilidad del experimento final

Las rutas del inventario son relativas a reproduccion/. En el inventario siguiente, G designa `outputs/local_runs/generation_scaling_v3/` e I, `outputs/local_runs/explicitness_v3/`. Los hashes completos se conservan en el informe documental citado.

Parejas y golden set

Directorio: `data/golden/variants/`.

Archivos: `explicitness_pairs_v3_all26.jsonl` y `generation_golden_v3.jsonl`.

Alcance: 26 preguntas base y 52 formulaciones, no 52 unidades independientes.

Evidencia

Fuente en I: `qrels_final_v3.jsonl`, `configuration.json` y `runs/`.

Alcance: juicios por pregunta base y cuatro rankings. Se conserva la procedencia de consenso, arbitraje y decisión humana.

Respuestas

Fuente en G: doce archivos `rag_*_top5_*.jsonl`, metadatos y registros de fallos de los tres generadores incluidos.

Alcance: 604 respuestas válidas y veinte fallos finales separados.

Evaluaciones excluidas

Directorios en G: `eval_invalida_solo_ids_20260927/` y `eval_diagnostico_v3.1_parcial/`.

Alcance: pases archivados que no alimentan el análisis final.

Jueces válidos

Archivos en G: `eval_verdicts_v3.jsonl` y `eval_verdicts_v3_juezB.jsonl`.

Alcance: A registra gpt-6-sol/openai-codex y B, mimo-v2.6-pro/opencode-go. Ambos declaran la rúbrica v3.2, un instrumento ordinal propio, no métricas ejecutadas mediante la biblioteca RAGAS.

Arbitraje y validación humana

Directorio en G: `arbitraje_discrepancias_v3/`.

Archivos: `arbitraje_v3.jsonl` (propuesta delegada de gpt-6-astra) y `validacion_humana_v3.jsonl` (decisiones confirmadas por el autor).

Alcance: 158 celdas y 233 criterios discrepantes. El acta contiene una fila adicional de convención metodológica que no cuenta como caso. La propuesta del árbitro no se presenta por sí sola como adjudicación humana.

Consolidación

Archivos en G: `eval_final_v3.jsonl` y `eval_final_v3_summary.json`.

Alcance: 446 casos de consenso y 158 celdas con validación humana en al menos un criterio. La procedencia se conserva por criterio.

Análisis

Fuentes: `scripts/analysis_capacity_v3.py`, `scripts/analyze_final_experiment.py` y los destinos de corrección identificados en D.

Alcance: soporte común por celda, bootstrap por pregunta base y contenido desglosado por conducta esperada.

La evaluación remota puede prepararse offline con `scripts/evaluate_rag_remote_v3.py --only-batches`, aportando los doce archivos de generación, `--batches-dir`, `--output` y `--batch-size 10`. Retirar `--only-batches` invoca al proveedor y no forma parte del reanálisis offline.

No existe una CLI dedicada de consolidación RAG v3. El cierre histórico fue manual/asistido. Su reconstrucción exige aplicar, por clave completa y criterio, el consenso A=B o la decisión humana final. La validación comprueba esa regla sin producir nuevas adjudicaciones.

El pre-registro se conserva con sus diez enmiendas. Entre plan y ejecución cambiaron los generadores admitidos, el parser, el instrumento, la conducta y el número de réplicas bootstrap. El índice externo de capacidad fue aportado por el autor sin captura o exportación recuperable y no se usa como evidencia verificada ni como escala causal. Las identidades de jueces y árbitro son las registradas en ejecución, no una demostración de independencia

estadística. El juez B participó en el diseño de preguntas y A comparte proveedor con el árbitro. Ambos solapes son amenazas a la validez.

E.5. Corpus, figuras y render del apéndice

La adquisición inicial de PDF fue manual, desde las páginas oficiales. Los hashes del inventario permiten verificar los archivos de data/raw/openstax/. Reextraer fórmulas produciría un corpus distinto y exigiría revisar fragmentos, qrels y resultados. El corpus congelado no se regenera durante el reanálisis.

La regeneración de figuras es opcional y requiere R con dplyr, ggplot2, jsonlite, knitr y tibble. En el cuaderno 04 se activa cambiando REGENERAR_FIGURAS de False a True. También puede ejecutarse por separado:

```
uv run --no-sync python scripts/reproduce_figures.py \
  --run-id tribunal_01
```

El programa extrae literalmente los once bloques R de las fuentes finales de analista y sponsor y genera diez gráficos SVG en un destino nuevo. No modifica ni renderiza ninguna memoria. No se garantiza identidad binaria de los SVG entre entornos gráficos distintos.

La fotografía del entorno histórico está en docs/render_environment_20260930.json: R 4.6.1, dplyr 1.2.1, ggplot2 4.0.3, jsonlite 2.0.0 y knitr 1.51, junto con Quarto 1.8.25 y Typst incluido 0.13.0. Es un registro del entorno instalado, no un lock ni una prueba de instalación limpia. Este apéndice no ejecuta bloques R.

El PDF técnico se genera desde reproduccion/ y se coloca en documentos/ con el nombre de entrega. Este comando renderiza exclusivamente el apéndice:

```
DOC=tfm_v2_4_apendices_tecnicos.qmd
PDF=tfm_apendices_tecnicos_AndresGonzalezSanchez.pdf
quarto render "$DOC" --to typst --output "$PDF"
mv "$PDF" ../documentos/
```

Los cinco notebooks se han comprobado en una copia reubicada, con kernels nuevos y bloqueo desde Python de la lectura de los árboles originales y de las conexiones externas. Esa guarda no equivale a aislamiento del sistema operativo ni restringe R. No se ha verificado una instalación limpia de R ni repetido inferencia. Que un identificador contenga v1 no implica una dependencia de esa carpeta.